

Introduction to Visual Studio

Visual Studio is an **Integrated Development Environment(IDE)** developed by Microsoft to develop

Desktop applications,

GUI(Graphical User Interface),

console, web applications,

mobile applications,

cloud, and web services, etc.

With the help of this IDE, you can create managed code as well as native code. It uses the various platforms of Microsoft software development software like Windows store, Microsoft Silverlight, and Windows API, etc.

It is not a language-specific IDE as you can use this to write code in C#, C++, VB(Visual Basic), Python, JavaScript, and many more languages. It provides support for 36 different programming languages. It is available for Windows as well as for macOS.

Before .NET Framework



Problems in Pre .Net

1. VC++ Had Object Oriented and threading but complex.
2. VB6 Not Object Oriented and no threading but simple.
3. ASP Script based /Integrated ,Late Bound, Difficult to maintain and Debug , Not object oriented.
4. COM DLL Hell! (Mainly Versioning and Deploying)

Introduction to .NET Framework

The **.NET Framework** is a software development framework developed by Microsoft that provides a runtime environment and a set of libraries and tools for building and running applications on Windows operating systems.

The .NET framework is primarily used on Windows, while .NET Core (which evolved into just .NET starting from version 5) is cross-platform.

The framework supports multiple programming languages, such as C#, F#, and VB.NET, and supports a range of application types, including desktop, web, mobile, cloud, and gaming applications.

.Net Framework Features

- 1.Object oriented code.
- 2.Multiple Languages.
3. Multiple Platforms.
4. Multiple Project types(eg. Web based, desktop based,etc).
5. Better Security.
6. Improved performance.

.Net Core Features

1. Open-Source
2. Cross platforms
3. Lightweight
4. Extensible

.Net Core

1. Upto version 2.2 Supports ASP.Net MVC and Web APIs
2. Version 3+ supports Winforms and WPF.
3. Course cover version 5

Managed vs UnManaged Code

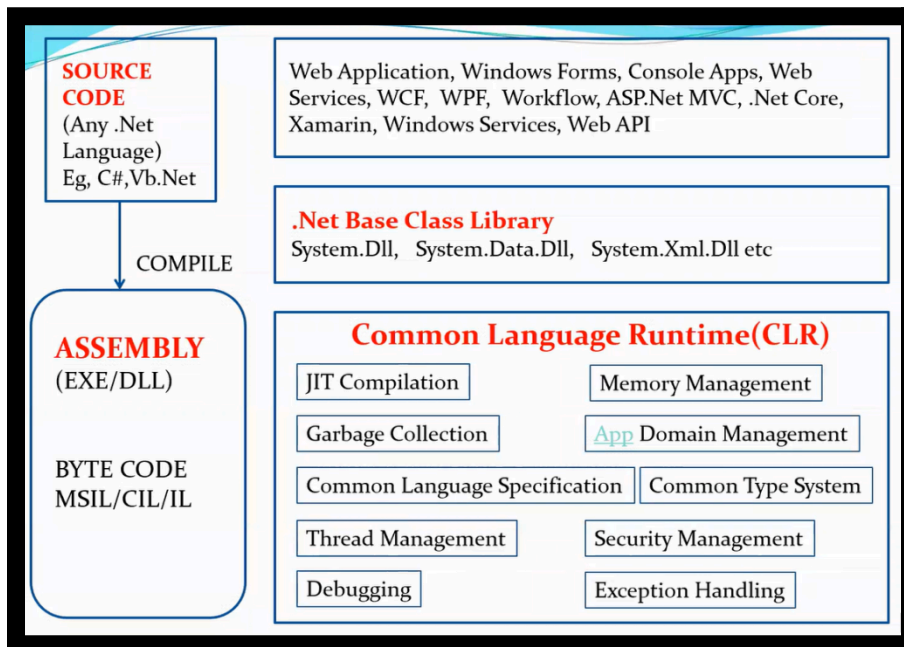
1. Managed code is run by CLR(All .net code).

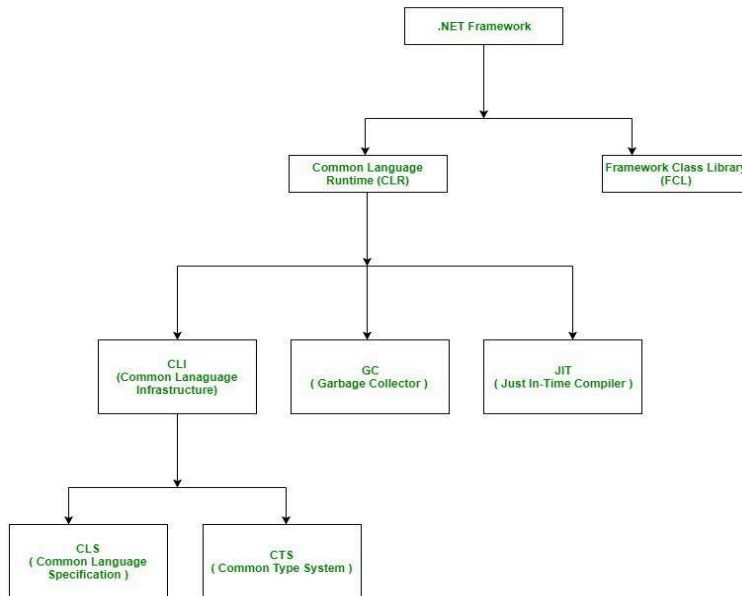
2. Unmanage code is not by CLR.

Egs. Windows DLLs(PInvoke) and COM Apps (using com Interops)

Main Components of .NET Framework

- **Common Language Runtime(CLR):**
 - o The CLR is the heart of the .NET Framework, acting as a virtual machine that runs the code and manages various services such as memory management, security, and thread management. Code that is compiled and executed within the CLR is called "Managed Code," while code that the CLR does not manage is known as "Unmanaged Code."
- **.NET Framework Class Library (FCL):**
 - o The FCL provides a large set of reusable classes and methods for application development.
 - o This includes libraries for input/output operations, networking, data access, UI controls, and more.





.NET Programming Languages

The .NET ecosystem supports various programming languages developed by Microsoft and third-party contributors. Some of the most widely used languages are:

- **C# .NET:** A modern, object-oriented language used for a wide range of application types.
- **VB.NET:** A language designed for ease of use, with syntax similar to traditional Basic.
- **F# .NET:** A functional-first programming language, well-suited for complex algorithms and data processing.
- **C++ .NET:** An extension of C++ designed for managed code applications.
- **J# .NET:** A language that provides .NET compatibility for Java developers.
- **IronRuby, IronPython:** .NET implementations of the Ruby and Python languages.
- **Other Languages:** JScript.NET, C Omega, ASML, and more.

Advantages of .NET Framework

- **Multi-Language Support:** The .NET Framework supports several programming languages, including C#, F#, and Visual Basic, allowing developers to choose the language that best suits their needs while still benefiting from the same set of libraries and tools.
- **Cross-Platform Compatibility (via .NET Core and Mono):** Although initially designed for Windows, the .NET Framework can now run on various operating systems through .NET Core and Mono, making it a versatile solution for cross-platform development.
- **Large and Active Developer Community:** The .NET Framework benefits from a broad developer base, creating a rich ecosystem of libraries, tools, and resources. This helps developers solve problems more easily and share solutions across the community.

- **Security Features:** The framework includes various security features such as code access security and digital signatures, helping protect applications from malicious threats and vulnerabilities.
- **Productivity:** The set of pre-built libraries and tools offered by .NET accelerates development time, enabling developers to focus on business logic rather than reinventing common functionalities.

Intermediate Language (IL)

Intermediate Language (IL), also known as Common Intermediate Language (CIL), is a low-level, platform-independent language that serves as a middle ground between high-level programming languages and machine code. It's a crucial part of the .NET framework, where code written in languages like C# or VB.NET is compiled into IL, which is then converted to machine code just before execution by the Just-In-Time (JIT) compiler.

CLR and its functions

- JIT Compilation
- Memory Management
- Garbage Collection
- AppDomain Management
- Memory Management
- CLS, CTS
- Security

JIT Compilation

Just-in-time (JIT) compilation is a method where code is compiled at runtime, rather than before execution, like a traditional compiler. This approach, often used in interpreters and run-time environments, translates bytecode (or a similar intermediate representation) into machine code during program execution. JIT compilation can improve performance by optimizing frequently executed code, especially in areas like loops and hot spots,

Memory Management

Memory management is the process of controlling and coordinating a computer's main memory, ensuring efficient allocation and de allocation of memory space for operating systems, applications, and other processes. It involves allocating memory to processes that need it and freeing it up when it's no longer in use.

Key aspects of memory management:

- **Allocation:** Assigning memory space to programs or processes that need it.
- **De allocation (Freeing):** Releasing memory space back to the system when it's no longer in use.
- **Tracking:** Keeping track of all memory locations, whether allocated or free.

C# .NET Framework

C# (C-Sharp) is a modern, object-oriented programming language developed by Microsoft in 2000.

It is a part of the .NET ecosystem and is widely used for building desktop, web, mobile, cloud, and enterprise applications.

This is originally tied to the .NET Framework, C# has evolved to be the primary language used across the .NET platform which now supports cross-platform development.

The **.NET Framework** initially launched in 2002 and it has evolved into a broader and more powerful ecosystem known as .NET 5 and beyond.

This platform combines the best of .NET Framework, .NET Core, and Xamarin and it enables the developers to build applications that run on Windows, Linux, and macOS.

- C# originates from the **C programming** family and shares similarities with other widely-used languages like **C++** and **Java**.
- Mainly used to develop the desktop apps, windows apps, games(with **unity engine**), mobile apps and web services etc.
- C# is the primary language for developing games using the Unity engine.
- With **.NET Core**, C# applications can run on Windows, macOS, and Linux.
- The latest version, **C# 13** was released in November 2024 alongside **.NET 9**.

Key Components of the .NET Framework

The .NET Framework's basic architecture consists of two key elements:

1. Common Language Runtime (CLR): The Common Language Runtime (CLR) is the core runtime engine of the .NET Framework, responsible for managing the execution of code written in any of the .NET-supported languages, such as C#, F#, and VB.NET. It provides various services, including:

- **Memory Management:** Automatic memory allocation and garbage collection.
- **Code Safety:** Ensures type safety and handles security through Code Access Security (CAS).
- **Execution Management:** Converts Intermediate Language (IL) code into machine code via the Just-In-Time (JIT) compiler.

The CLR enables Cross-Language Interoperability, which allows components written in different languages to work together seamlessly.

2. .NET Framework Class Library (FCL): The .NET Framework Class Library (FCL) is a vast collection of pre-built functions and classes available to developers. The FCL provides a range of namespaces that contain classes for tasks like:

- File I/O
- Networking
- Database access
- Graphical User Interface (GUI) design

Additionally, it includes tools such as the Visual Studio IDE, which simplifies application development, debugging, and testing.

3. Other Key Components:

- **Common Type System (CTS):** It ensures that types are consistent across different languages within the .NET ecosystem.
- **Common Intermediate Language (CIL):** This is a low-level language that is compiled from source code and later converted into machine code by the JIT compiler.
- **Assemblies:** The bundles of code that are versioned and deployed together, providing the essential building blocks of a .NET application.

Console Applications and Class Libraries.Net Core

In .NET Core, a class library is a reusable set of code that defines types and methods used by applications, while a console application is a runnable program that interacts with the user through a command-line interface. Class libraries can be used by multiple applications, while console applications are standalone programs.

Class Library:

- **Definition:**

A collection of reusable code, typically organized into classes and methods, that can be referenced by other applications.

- **Purpose:**

To encapsulate functionality and provide a modular approach to development.

- **Example:**

A library that provides common string manipulation functions or data validation routines.

- **Usage:**

You can create a class library project in your .NET solution and then add a reference to it in other projects, such as a console application.

- **Building:**

Class libraries do not have a Main method and are typically compiled into a DLL file.

Console Application:

- **Definition:** A program that interacts with the user through a command-line interface.
- **Purpose:** To perform tasks that are often automated or require user input via text.
- **Example:** A program that reads data from a file, performs some calculations, and then prints the results to the console.
- **Usage:** Console applications have a Main method where the program's execution begins.
- **Building:** Console applications are typically compiled into an EXE file.

In essence:

- Class libraries provide reusable code that other applications can use.
- Console applications are standalone programs that interact with the user through the command line.

What is C#?

C# is pronounced "C-Sharp".

It is an object-oriented programming language created by Microsoft that runs on the .NET Framework.

C# has roots from the C family, and the language is close to other popular languages like [C++](#) and [Java](#).

The first version was released in year 2002. The latest version, **C# 13**, was released in November 2024.

C# is used for:

- Mobile applications
- Desktop applications
- Web applications
- Web services
- Web sites
- Games
- VR
- Database applications
- And much, much more!

Why Use C#?

- It is one of the most popular programming languages in the world
- It is easy to learn and simple to use
- It has huge community support
- C# is an object-oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs
- As C# is close to [C](#), [C++](#) and [Java](#), it makes it easy for programmers to switch to C# or vice versa

Data Types in .net and CTS equivalents

Data types specify the type of data that a valid C# variable can hold. C# is a **strongly typed programming language** because in C# each type of data (such as integer, character, float, and so forth) is predefined as part of the programming language and all constants or variables defined for a given program must be described with one of the data types.

Data Types in C# is Mainly Divided into 3 Categories:

- **Value Data Types**
- **Reference Data Types**
- **Pointer Data Type**

1. Value Data Types

In C#, the Value Data Types will directly store the variable value in memory and it will also accept both signed and unsigned literals. The derived class for these data types are **System.ValueType**. Following are **different Value Data Types** in C# programming language

Signed & Unsigned Integral Types

There are 8 integral types which provide support for 8-bit, 16-bit, 32-bit, and 64-bit values in signed or unsigned form.

Alias	Type Name	Type	Size(bits)	Range	Default Value
sbyte	System.Sbyte	signed integer	8	-128 to 127	0
short	System.Int16	signed integer	16	-32768 to 32767	0
Int	System.Int32	signed integer	32	-231 to 231-1	0
long	System.Int64	signed integer	64	-263 to 263-1	0L
byte	System.byte	unsigned integer	8	0 to 255	0
ushort	System.UInt16	unsigned integer	16	0 to 65535	0
uint	System.UInt32	unsigned integer	32	0 to 232	0

Alias	Type Name	Type	Size(bits)	Range	Default Value
ulong	System.UInt64	unsigned integer	64	0 to 263	0

Floating Point Types

There are **2 floating point data types** which contain the decimal point.

Alias	Type name	Size(bits)	Range (aprox)	Default Value
float	System.Single	32	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	0.0F
double	System.Double	64	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	0.0D

- **Float:** It is **32-bit single-precision** floating point type. It has 7 digit Precision. To initialize a float variable, use the suffix f or F. Like, float x = 3.5F;. If the suffix F or f will not use then it is treated as double.
- **Double:** It is **64-bit double-precision** floating point type. It has 14 - 15 digit Precision. To initialize a double variable, use the suffix d or D. But it is not mandatory to use suffix because by default floating data types are the double type.

Decimal Types

The decimal type is a 128-bit data type suitable for financial and monetary calculations. It has 28-29 digit Precision. To initialize a decimal variable, use the suffix m or M. Like as, decimal x = 300.5m;. If the suffix m or M will not use then it is treated as double.

Alias	Type name	Size(bits)	Range (aprox)	Default value
decimal	System.Decimal	128	$\pm 1.0 \times 10^{-28}$ to $\pm 7.9228 \times 10^{28}$	0.0M

Character Types

The character types represents a UTF-16 code unit or represents the 16-bit Unicode character.

Alias	Type name	Size In(Bits)	Range	Default value
char	System.Char	16	U +0000 to U +ffff	'\0'

Boolean Types

It has to be assigned either true or false value. Values of type bool are not converted implicitly or explicitly (with casts) to any other type. But the programmer can easily write conversion code.

Alias	Type Name	Possible Values
bool	System.Boolean	true / false

Reference Data Types

The Reference Data Types will contain a memory address of variable value because the reference types won't store the variable value directly in memory. When you create a reference type variable, such as an object or a string, you are actually storing a reference (or pointer) to the location in memory where the data is held. The actual data for reference types is stored on the heap. The heap is a large pool of memory used for dynamic memory allocation. The built-in reference types are **string**, **object**.

String

It represents a sequence of Unicode characters and its type name is **System.String**. So, string and String are equivalent.

Object

In C#, all types, predefined and user-defined, reference types and value types, inherit directly or indirectly from Object. So basically it is the base class for all the data types in C#. Before assigning values, it needs type conversion. When a variable of a value type is converted to object, it's called **boxing**. When a variable of type object is converted to a value type, it's called **unboxing**. Its type name is **System.Object**.

Pointer Data Type

The Pointer Data Types will contain a memory address of the variable value. To get the pointer details we have a two symbols **ampersand (&)** and **asterisk (*)**.

- **ampersand (&):** It is known as Address Operator. It is used to determine the address of a variable.
- **asterisk (*):** It also known as Indirection Operator. It is used to access the value of an address.

